

Strategy Pattern

Course

**CSE-3104: Software Engineering &
Information System Lab**

Prepared By:

Team 1 (Innovex)

Instructor:

**Md. Samsuddoha
Assistant Professor
University of Barishal**

TABLE OF CONTENTS

01	Introduce the pattern & Intent
02	Problems
03	Solution
04	Structure
05	Examples & Real Analogy
06	Advantage & Disadvantage
07	Application
08	Coding

What is Design Pattern?

Design Pattern

A Design Pattern is a reusable and proven blueprint that provides a general solution to commonly occurring software design problems. It is not ready-made code but a customizable template that developers adapt according to their needs.

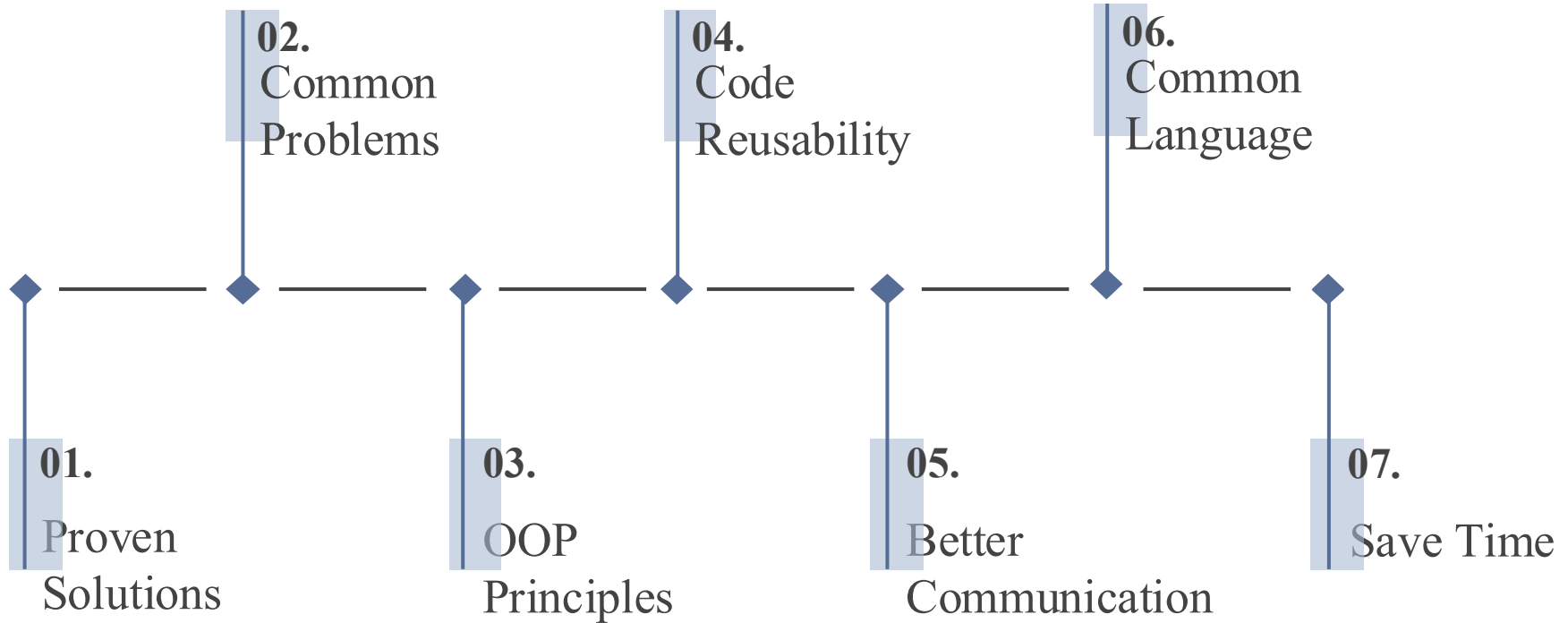
General Concept

Not actual code, just an idea or structure.

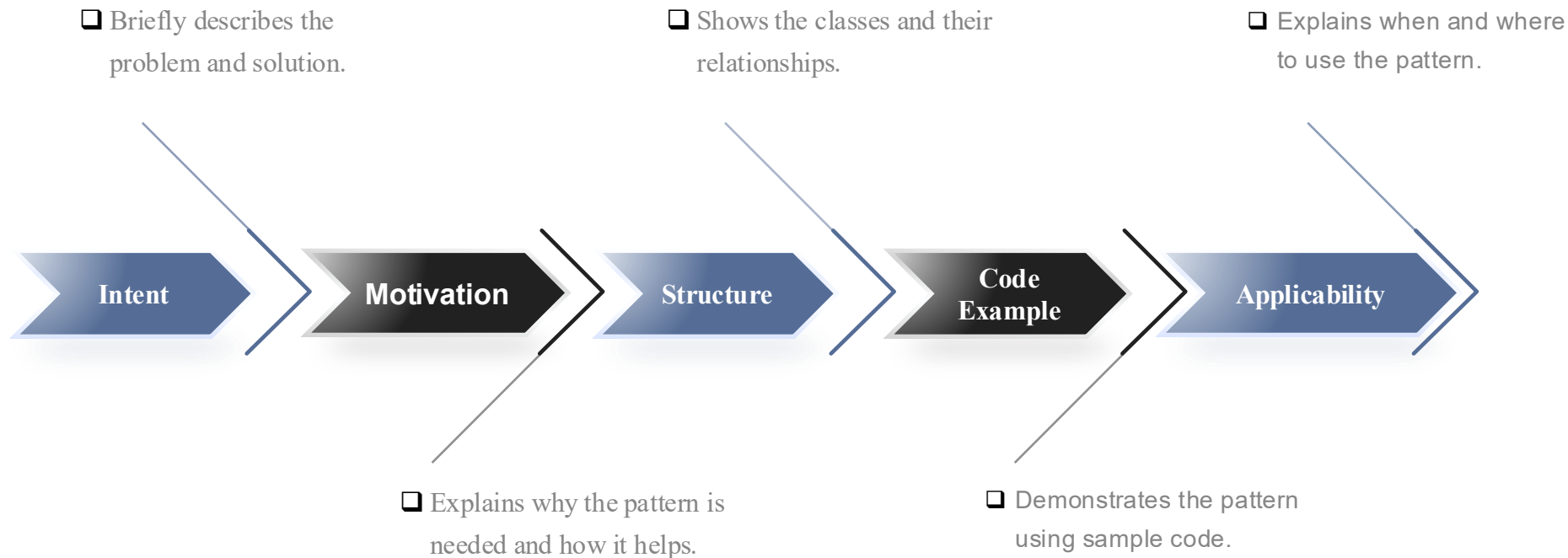
Customizable

Can be adapted to different programs

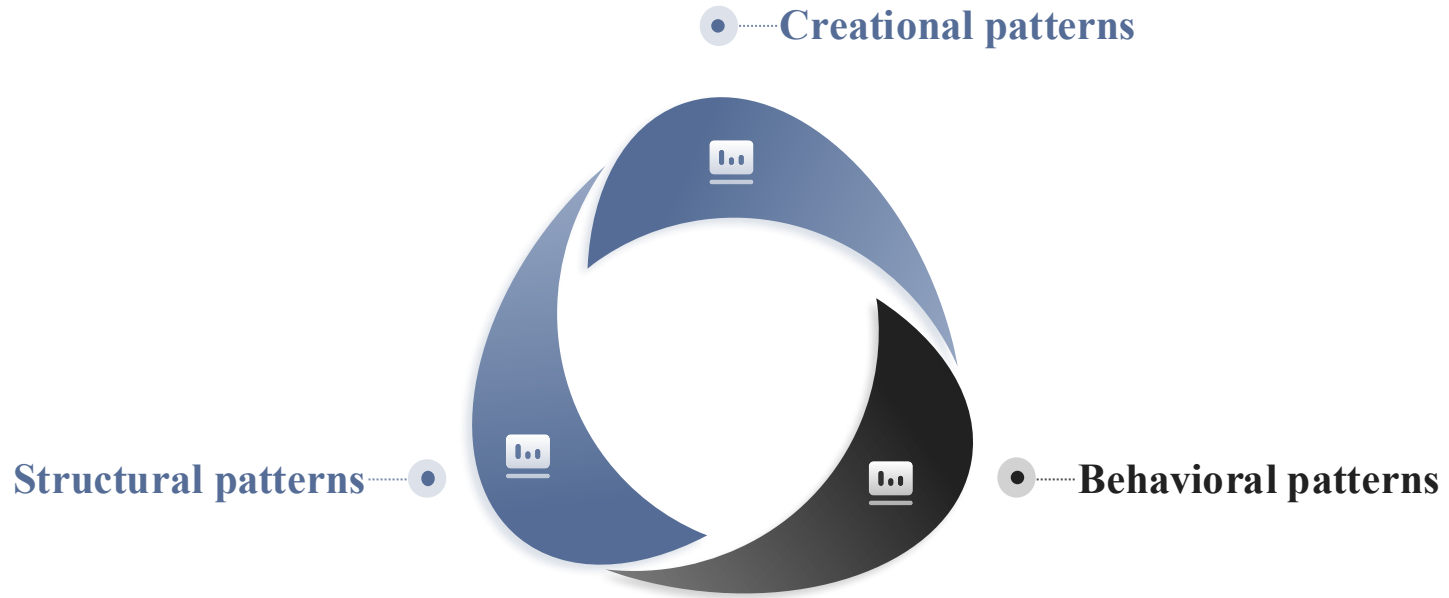
Why Should You Learn Design Patterns?



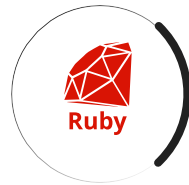
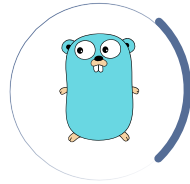
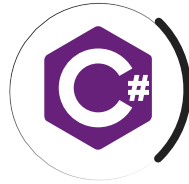
What Does a Design Pattern Consist Of?



Classification of Design Patterns



Design Patterns in different programming languages



What is Strategy Design Pattern?

Strategy Design Pattern

Strategy Design Pattern is a behavioral design pattern that allows you to define multiple algorithms (strategies), place each one in a separate class, and switch between them at runtime without changing the main code.

Example

A **Navigator App** can have different routing strategies:

Car Route

Walking Route

Cycling Route

Public Transport Route

Instead of writing all route logic inside one class, each route becomes a separate strategy and the app selects one when needed.

Problem Statement

Navigator app initially supported car routes only.

New routing options includes:



Issues Faced

Issue-2

Diffult to maintain
and modify

Issue-1

Class became large
and complex

Issue-3

Higher risk of bugs

Issue-4

Frequent merge conflicts
in team development

Solution

Extract each routing algorithm into a separate Strategy class.

Define a common interface: buildRoute(start, end)

Examples

CarRouteStrategy

CyclingRouteStrategy

WalkingRouteStrategy

Navigator – delegates route calculation to the selected strategy

PublicTransportStrategy

Strategies can be switched at runtime

Benefits

✓ Easy to add new routing algorithms

✓ Better maintainability

✓ Follows the Open/Closed Principle

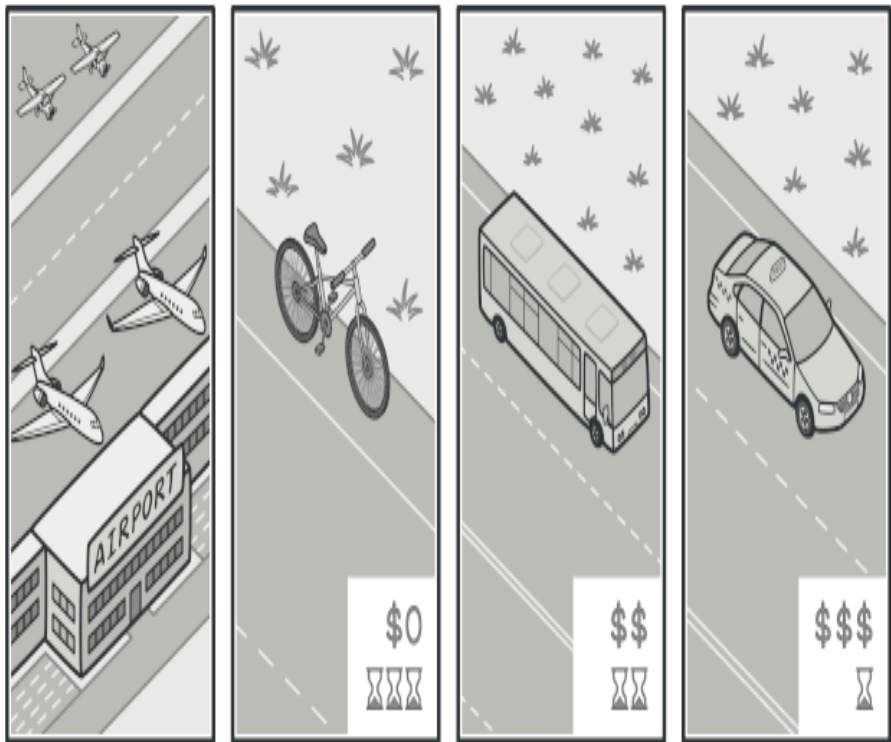
✓ Reduced code duplication and bugs

✓ Improved teamwork and scalability

Getting to the Airport

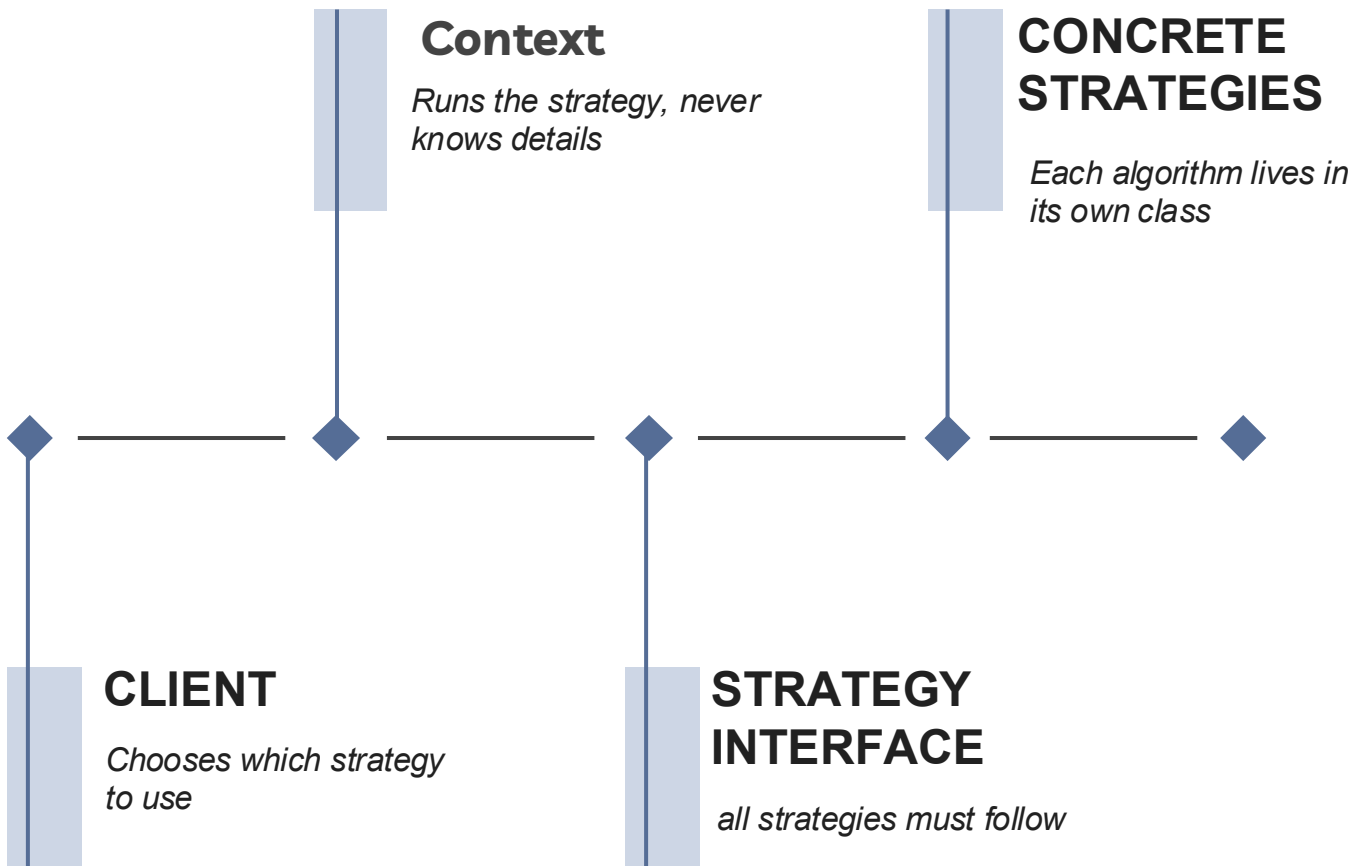
Transportation strategies

Depending on factors

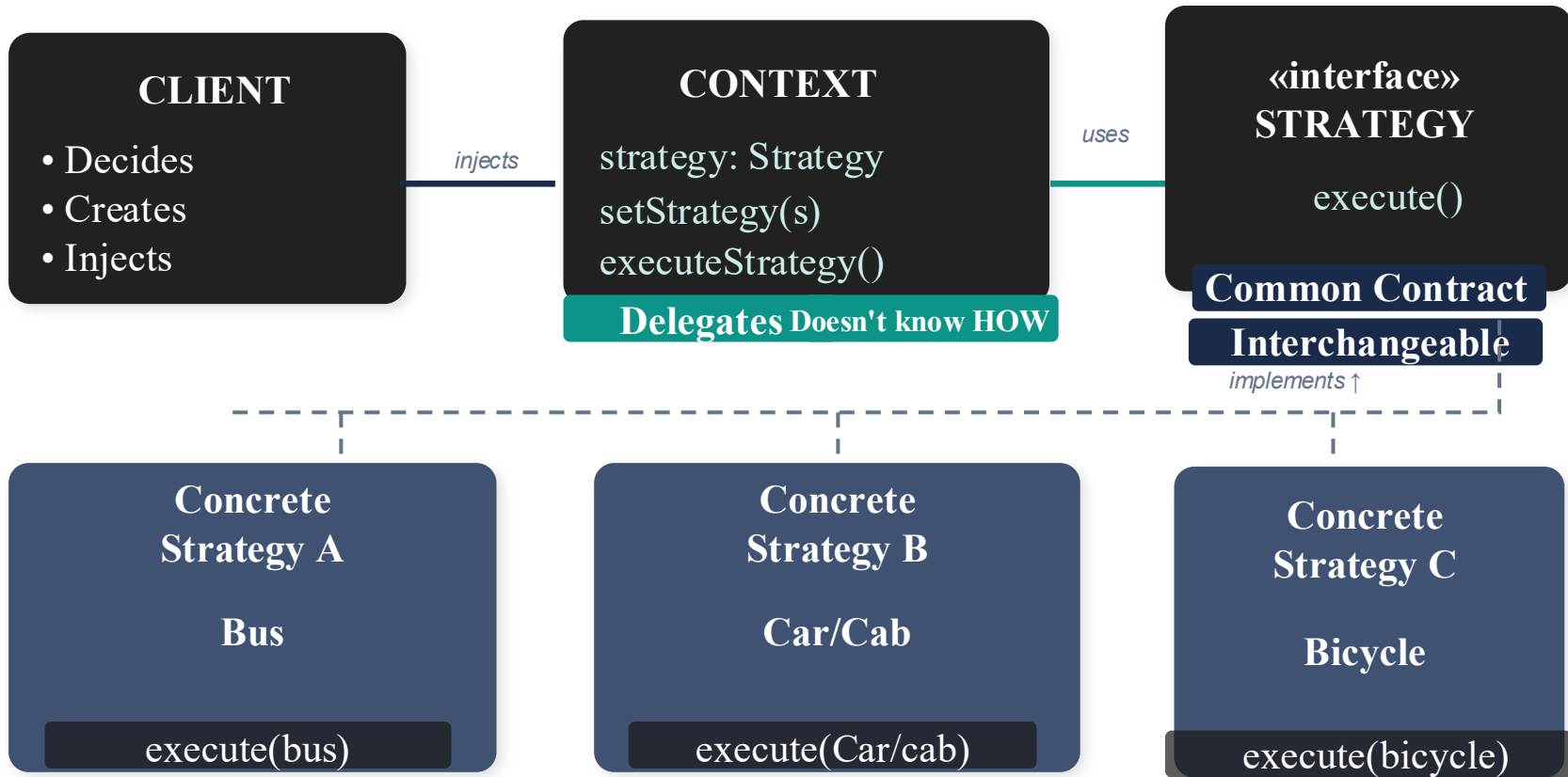


Real Life Analogy

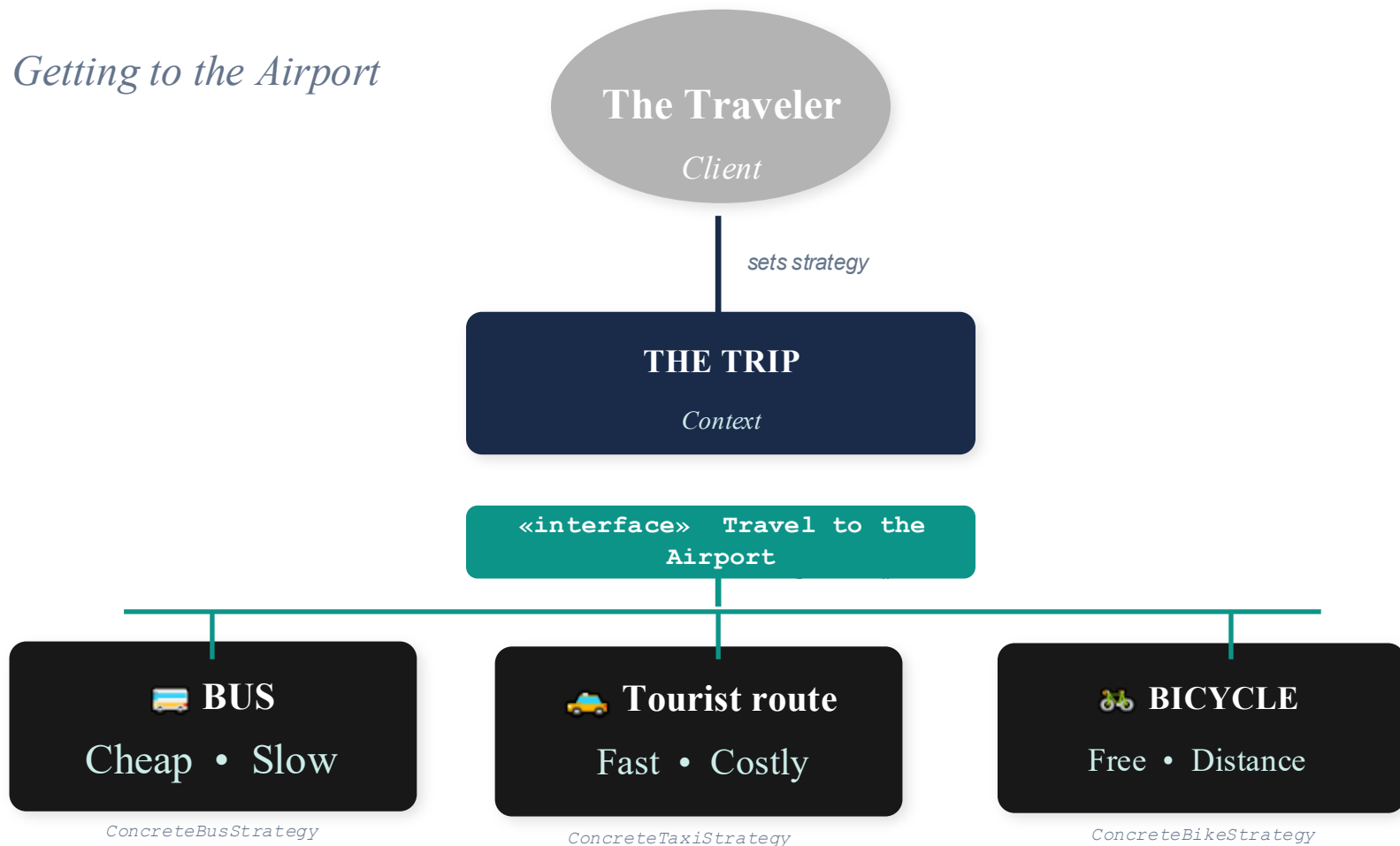
Structure - 4 Components



Structure



✈ Getting to the Airport



Advantages

1. Runtime Flexibility

4. Code Reusability

2. Open / Closed Support

5. Less Conditional Logic

3. Easy Maintenance

6. Easy Testing

Disadvantages

1. More Classes

2. Extra Overhead

3. Strategy Selection Complexity

4. Overengineering Risk

5. More Object Interaction

When to Use ?

**Multiple Solutions
Exist**

**Dynamic Behavior
Needed**

**Remove Complex
Conditions**

Need Scalability

**Apply SOLID
Principles**

Code

```
1 import java.util.Arrays;
2 import java.util.List;
3 import java.util.Scanner;
4
5 // — Strategy Interface —
6
7 interface RouteStrategy {
8     List<String> buildRoute(String origin, String destination);
9 }
10
11 // — Concrete Strategies —
12
13 class CarRouteStrategy implements RouteStrategy {
14     public List<String> buildRoute(String origin, String destination) {
15         System.out.println(" [Car] Building fastest road route...");
16         return Arrays.asList(origin, "Express Highway", "City Flyover", destination);
17     }
18 }
19
20 class WalkingRouteStrategy implements RouteStrategy {
21     public List<String> buildRoute(String origin, String destination) {
22         System.out.println(" [Walking] Building pedestrian-friendly route...");
23         return Arrays.asList(origin, "Central Park", "River Walk", destination);
24     }
25 }
26
27 class PublicTransitRouteStrategy implements RouteStrategy {
28     public List<String> buildRoute(String origin, String destination) {
29         System.out.println(" [Transit] Building public transport route...");
30         return Arrays.asList(origin, "Bus Stop", "Metro Station", destination);
31     }
32 }
33
34 class CyclingRouteStrategy implements RouteStrategy {
35     public List<String> buildRoute(String origin, String destination) {
36         System.out.println(" [Cycling] Building bike-friendly route...");
37         return Arrays.asList(origin, "Cycle Track", "Green Lane", destination);
38     }
39 }
```

```
41 class TouristRouteStrategy implements RouteStrategy {
42     public List<String> buildRoute(String origin, String destination) {
43         System.out.println(" [Tourist] Building scenic route...");
44         return Arrays.asList(origin, "City Museum", "Historic Bridge", "View Point", destination);
45     }
46 }
47
48 // — Context —
49
50 class Navigator {
51
52     private RouteStrategy strategy;
53
54     public Navigator(RouteStrategy strategy) {
55         this.strategy = strategy;
56     }
57
58     public void setStrategy(RouteStrategy strategy) {
59         this.strategy = strategy;
60     }
61
62     public void navigate(String origin, String destination) {
63         System.out.println("\n From : " + origin);
64         System.out.println(" To : " + destination);
65         List<String> route = strategy.buildRoute(origin, destination);
66         System.out.println(" Route: " + String.join(" -> ", route));
67         System.out.println(" " + "-".repeat(50));
68     }
69 }
70
71
72
73 public class NavigationApp {
74
75     public static void main(String[] args) {
76
77         Scanner input = new Scanner(System.in);
78     }
```

```

73 public class NavigationApp {
74
75     Run | Debug
76     public static void main(String[] args) {
77
78         Scanner input = new Scanner(System.in);
79
80         String[] places = {
81             "Hotel Central",
82             "Airport",
83             "University",
84             "City Mall"
85         };
86
87         System.out.println("=".repeat(count: 55));
88         System.out.println(x: "    NAVIGATOR APP - Strategy Pattern");
89         System.out.println("=".repeat(count: 55));
90
91         System.out.println(x: "\nChoose Starting Point:");
92         for (int i = 0; i < places.length; i++) {
93             System.out.println((i + 1) + ". " + places[i]);
94         }
95         System.out.print(s: "\nEnter choice: ");
96         int start = input.nextInt();
97
98         System.out.println(x: "\nChoose Destination:");
99         for (int i = 0; i < places.length; i++) {
100             System.out.println((i + 1) + ". " + places[i]);
101         }
102         System.out.print(s: "\nEnter choice: ");
103         int end = input.nextInt();
104
105         String from = places[start - 1];
106         String to = places[end - 1];
107
108         System.out.println(x: "\nChoose Travel Mode:");
109         System.out.println(x: "1. Car - fastest road route");
110         System.out.println(x: "2. Walking - pedestrian-friendly");
111         System.out.println(x: "3. Transit - public transport");
112         System.out.println(x: "4. Cycling - bike-friendly");
113         System.out.println(x: "5. Tourist - scenic route");
114         System.out.print(s: "\nEnter choice: ");
115         int modeChoice = input.nextInt();
116
117         RouteStrategy strategy;
118         switch (modeChoice) {
119             case 1: strategy = new CarRouteStrategy(); break;
120             case 2: strategy = new WalkingRouteStrategy(); break;
121             case 3: strategy = new PublicTransitRouteStrategy(); break;
122             case 4: strategy = new CyclingRouteStrategy(); break;
123             case 5: strategy = new TouristRouteStrategy(); break;
124             default:
125                 System.out.println(x: "Invalid choice. Defaulting to Car.");
126                 strategy = new CarRouteStrategy();
127         }
128
129         // — Navigate —
130         Navigator navigator = new Navigator(strategy);
131
132         System.out.println("=".repeat(count: 55));
133         navigator.navigate(from, to);
134         System.out.println("=".repeat(count: 55));
135
136         input.close();
137     }

```

```

73 public class NavigationApp {
74     public static void main(String[] args) {
75         System.out.print(s: "\nEnter choice: ");
76         int start = input.nextInt();
77
78         System.out.println(x: "\nChoose Destination:");
79         for (int i = 0; i < places.length; i++) {
80             System.out.println((i + 1) + ". " + places[i]);
81         }
82         System.out.print(s: "\nEnter choice: ");
83         int end = input.nextInt();
84
85         String from = places[start - 1];
86         String to = places[end - 1];
87
88         System.out.println(x: "\nChoose Travel Mode:");
89         System.out.println(x: "1. Car - fastest road route");
90         System.out.println(x: "2. Walking - pedestrian-friendly");
91         System.out.println(x: "3. Transit - public transport");
92         System.out.println(x: "4. Cycling - bike-friendly");
93         System.out.println(x: "5. Tourist - scenic route");
94         System.out.print(s: "\nEnter choice: ");
95         int modeChoice = input.nextInt();
96
97         RouteStrategy strategy;
98         switch (modeChoice) {
99             case 1: strategy = new CarRouteStrategy(); break;
100             case 2: strategy = new WalkingRouteStrategy(); break;
101             case 3: strategy = new PublicTransitRouteStrategy(); break;
102             case 4: strategy = new CyclingRouteStrategy(); break;
103             case 5: strategy = new TouristRouteStrategy(); break;
104             default:
105                 System.out.println(x: "Invalid choice. Defaulting to Car.");
106                 strategy = new CarRouteStrategy();
107         }
108
109         // — Navigate —
110         Navigator navigator = new Navigator(strategy);
111
112         System.out.println("=".repeat(count: 55));
113         navigator.navigate(from, to);
114         System.out.println("=".repeat(count: 55));
115
116         input.close();
117     }

```

End of Lecture